e-Informatica Software Engineering Journal
Volume 00, Issue 0, pages: 1–35

BibTeX

Text-to-SQL Dataset Quality Assessment: The First Open-Source Validation Framework

Volodymyr Sabadosh*, Vladyslav Kotsovsky*
*Corresponding authors: vsababadosh@gmail.com, vladyslav.kotsovsky@uzhnu.edu.ua

Article info

Keywords:
Text-to-SQL system
dataset quality
LLM
SQL validation
semantic evaluation
empirical software engineering

Submitted: 1 Nov. 2024
Revised: 15 Jan. 2025
Accepted: 20 Feb. 2025
Available online: 1 Mar. 2025

Abstract

Background: LLM-based Text-to-SQL has advanced quickly, but benchmark and training datasets may contain defects that distort evaluation and fine-tuning.
Aim: Our objective is to develop and demonstrate a comprehensive automated system for evaluating Text-to-SQL dataset quality through multi-level validation addressing current gaps in systematic quality assessment.
Method: We implement a modular validation tool covering schema integrity, static SQL analysis, execution testing, antipattern detection, and semantic correspondence. Semantic correspondence is evaluated via an LLM-as-a-judge committee with majority voting. The system produces per-item annotations and persists all metrics in an analytical database, enabling direct querying and export as structured JSONL files or Markdown reports.
Results: Auditing a widely used benchmark reveals that surface checks can be misleading: near-perfect parsability and executability coexist with substantial hidden defects. The framework detects structural schema issues, large-scale referential-integrity violations, robustness/portability antipatterns, and join errors that change query semantics, with a non-trivial portion of items exhibiting semantic mismatch or ambiguity.
Conclusion: Multi-layer validation exposes dataset defects missed by executability-based checks. We release the open-source framework and a prioritized remediation roadmap for a popular Text-to-SQL benchmark.

1. Introduction

Text-to-SQL systems, which transform natural language queries into structured SQL queries, are experiencing a period of rapid development driven primarily by remarkable progress in Large Language Models (LLMs), as well as fundamental scientific interest and practical industry needs.

From a scientific perspective, the Text-to-SQL task serves as an important test of LLMs' capacity for structured reasoning and simultaneous understanding of natural language semantics and formal programming languages [1,2]. Recent achievements in this area demonstrate impressive results: systems using GPT-4 have reached 86.6% execution accuracy [3] on the Spider 1.0 leaderboard [4], and 80.88% [5] on the BIRD leaderboard [6]—approximately

© 2025 The Authors. Published by Wrocław University of Science and Technology Publishing House.
This is an open access article under the CC BY-SA 4.0 license international.

7-8× higher compared to results from 2018 [7], when the best non-LLM approaches (Seq2Seq/SQLNet/TypeSQL) on Spider 1.0 showed only ∼10% accuracy.

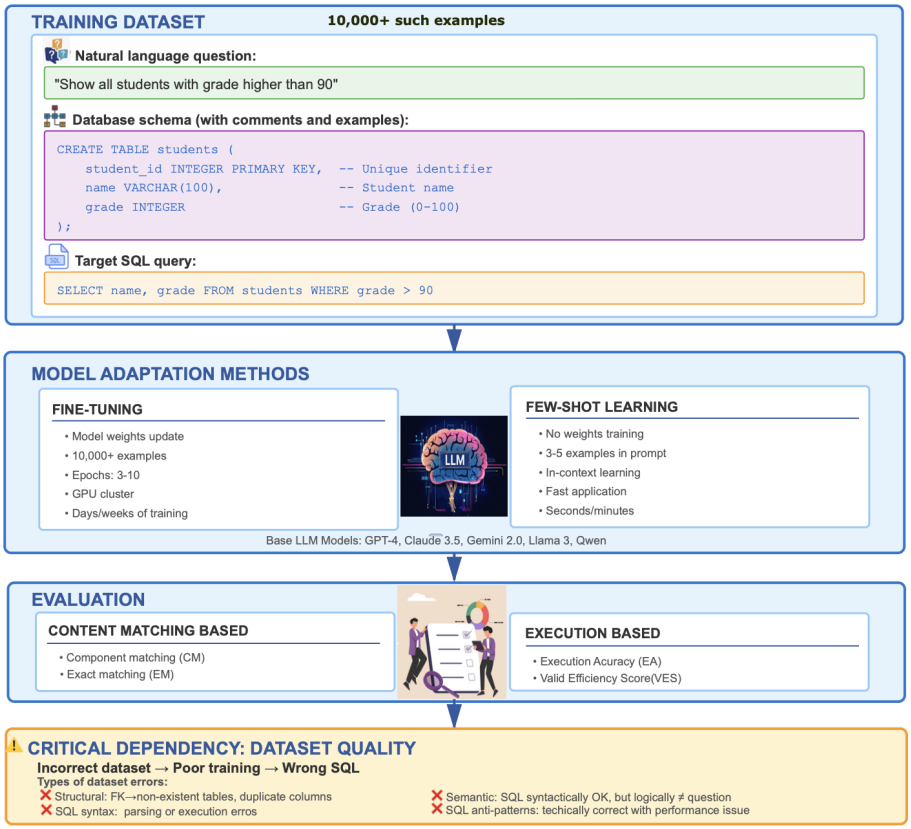

Figure 1. Complete training pipeline for LLMs in Text-to-SQL systems

From a practical standpoint, Text-to-SQL LLM systems are being actively deployed by leading cloud platforms. Snowflake Copilot can generate SQL from natural language in Snowsight [8]; Databricks AI/BI Genie converts plain English questions into equivalent SQL queries [9]; Microsoft Copilot in Fabric/Azure SQL supports Natural Language → SQL in SQL Database workspaces [10]. This lowers the barrier for business analysts and non-technical users, providing data access without knowledge of SQL syntax.

The typical process of training LLMs for Text-to-SQL tasks includes three key components (Figure 1): a training dataset containing triplets of (natural language question, database schema, target SQL query), an adaptation method (fine-tuning or few-shot learning) and evaluation (content matching and execution based). During fine-tuning, models adjust their weights on large Text-to-SQL datasets to better capture the mapping between natural language and SQL. In few-shot settings, by contrast, the model relies on in-context learning, using only a handful of examples in the prompt without updating its parameters. In both cases, dataset quality is critical: in fine-tuning, even a small fraction of flawed examples among tens of thousands can introduce systematic errors, while in few-shot prompting each individual example directly shapes model behavior, so a single incorrect pair may lead to consistently wrong generations (see Section 2.2). Consider specific types of errors that may be present in datasets:

- **Database schema structural errors**: incorrect foreign keys (references to non-existent tables/columns), duplicate columns, multiple primary keys, unknown data types.
- **SQL syntax/execution errors**: queries that cannot be parsed or executed on corresponding databases.
- **Semantic mismatches**: SQL queries that are syntactically correct but logically do not correspond to the natural language question (e.g., using SUM instead of AVG, incorrect JOIN conditions, missing necessary filters).
- **SQL anti-patterns**: technically correct queries with performance or style issues (SELECT *, absence of LIMIT, functions in WHERE that block index usage).

Problem statement

Despite the central role of datasets, most work on Text-to-SQL focuses on model architectures, prompting techniques and decoding strategies. Until recently, the quality of popular benchmarks was typically assumed rather than systematically measured. Recent studies have begun to scrutinize benchmark quality more directly, but existing studies either provide high-level descriptive statistics, focus on semantic label auditing for selected subsets, or concentrate on runtime validation of single generated queries rather than auditing entire corpora (see Section 2 for a detailed discussion). Consequently, researchers and practitioners still have limited visibility into structural defects, semantic mismatches, or stylistic anti-patterns hidden in widely used Text-to-SQL datasets. Therefore, our central research questions are:

- **RQ1**: How can we systematically evaluate the overall quality of Text-to-SQL datasets?
- **RQ2**: How accurate and internally consistent are popular Text-to-SQL benchmarks that serve simultaneously as training and evaluation data?

Research objective

This paper addresses this question by proposing a comprehensive validation framework for Text-to-SQL datasets and using Spider 1.0 as the primary end-to-end testbed for the framework. Our work is positioned within empirical software engineering: we treat benchmark datasets as software artefacts that should be inspected, measured and improved with the same rigour as source code or test suites.

Contribution statement

The main contributions of this paper are:

1. A novel multi-layer validation architecture for Text-to-SQL datasets integrating DDL schema validation, syntax analysis, execution testing, antipattern detection, and LLM-based semantic verification.
2. An open-source implementation of the validation framework, publicly available to enable reproducible quality assessment of existing and future Text-to-SQL benchmarks.
3. The first comprehensive empirical audit of all three Spider 1.0 partitions (11,840 examples in total) across five complementary quality dimensions.
4. Concrete empirical evidence that widely-used benchmarks contain significant quality issues invisible to standard evaluation metrics, including concentrated referential integrity catastrophes, substantial schema heterogeneity, and systematic semantic ambiguities.

How the paper is structured

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes our dataset auditing framework and analyzers. Section 4 reports empirical results. Section 5 discusses implications and threats to validity. Section 6 concludes and outlines future work.

2. Related Work

We review prior work across three complementary dimensions: the evolution of Text-to-SQL benchmarks, empirical evidence on data quality's impact on model performance, and existing approaches to dataset quality assessment.

2.1. Text-to-SQL Benchmarks and Datasets

The development of Text-to-SQL systems has been fundamentally shaped by the evolution of benchmark datasets. Early large-scale efforts, particularly WikiSQL [13], provided a foundational testbed with 80,654 natural-language questions and corresponding SQL queries over 24,241 Wikipedia tables. While WikiSQL enabled initial data-driven experimentation at scale, its design imposed significant constraints: queries were limited to single tables with a restricted SQL grammar excluding JOINs, nested subqueries, and complex aggregations. These limitations hindered the evaluation of compositional reasoning and realistic multi-table relational scenarios.

The release of Spider [7] at EMNLP 2018 marked a pivotal advancement in the field. Spider comprises 10,181 natural-language questions mapped to 5,693 unique SQL queries across 200 databases spanning 138 diverse domains. The dataset was meticulously constructed and validated by 11 Yale University students, ensuring rigorous annotation quality. A consolidated update published in 2020 resolved annotation inconsistencies and schema discrepancies, establishing the canonical Spider 1.0 version that remains the community standard. Spider's prominence derives from several key attributes: systematic human validation ensuring semantic correctness; balanced distribution across domains and difficulty levels; realistic multi-table schemas with extensive foreign-key relationships reflecting real-world database design; comprehensive coverage of SQL constructs including nested queries, set operations, aggregation with grouping, and multi-hop reasoning patterns. As of 2025, Spider has become the most widely adopted benchmark in the field, with over 700 citations across major academic indexing services.

Complementary benchmarks have emerged to address specific evaluation dimensions not fully captured by Spider. The BIRD benchmark [14] targets large-scale, content-rich databases (approximately 33 GB total), introducing practical challenges such as noisy data values, complex entity resolution requirements, and external knowledge dependencies. BIRD provides 12,751 question–SQL pairs across 95 substantial databases covering 37 professional domains, making it particularly valuable for assessing system robustness under realistic production conditions with imperfect data quality and ambiguous terminology.

Synthetic data generation has emerged as a scalable approach to augment training corpora. Gretel's synthetic Text-to-SQL dataset [15] (Gretel-Synth) offers over 100,000 high-quality examples spanning multiple business domains, while OmniSQL [16] employs a

multi-stage generation pipeline to produce more than 2.5 million training instances across 16,000+ synthetic databases, incorporating chain-of-thought reasoning annotations.

Despite the proliferation of alternative resources, Spider 1.0 remains the most comprehensive and widely adopted benchmark due to its unique combination of scale, structural diversity, annotation reliability, and faithful representation of relational complexity. However, even carefully curated benchmarks like Spider and BIRD are not immune to annotation errors, ambiguous question formulations, or schema inconsistencies. Recent auditing work, most notably NL2SQL-BUGs [17], has reported non-trivial rates of semantic errors in the gold SQL labels of these datasets, raising concerns about the common assumption that benchmark annotations are error-free and motivating more systematic dataset-level validation. We return to these auditing efforts in more detail in Section 2.3.

2.2. Data Quality and Robustness in Text-to-SQL

Empirical evidence increasingly demonstrates that the quality of training and evaluation data has a direct and often nonlinear influence on the performance of Large Language Models (LLMs) in Text-to-SQL tasks.

The Dr.Spider study [11] provides one of the most rigorous analyses of this phenomenon. By applying controlled perturbations to questions, database schemas, and SQL queries, the authors showed that LLM performance degrades sharply under reduced data quality. On average, execution accuracy dropped by 14.0%, and under the most severe perturbations the degradation reached 50.7%, revealing that current models exhibit low robustness to inconsistencies or noise in the training and evaluation pipelines. Notably, errors involving schema semantics (e.g., incorrect foreign keys or column references) produced disproportionately large declines, underscoring the sensitivity of Text-to-SQL systems to structural dataset quality.

Complementary findings come from Snowflake's Arctic-Text2SQL study [12], which evaluated the effects of mixing high-quality benchmark data (Spider, BIRD) with various forms of synthetic data, including Gretel-Synth. The authors observed that introducing lower-quality or weakly aligned synthetic examples into training can reduce final model accuracy, even when the overall dataset size increases—highlighting an inverse scaling trend. Their ablation experiments showed that quality mismatches between datasets lead to unstable gradients, degraded reasoning traces, and incorrect operator selection in generated SQL queries. These results reinforce the observation that the benefits of larger datasets cannot compensate for inconsistencies, noise, or semantic drift in the data.

2.3. Dataset Quality Assessment Frameworks

Although dataset quality is critical for Text-to-SQL system performance, systematic validation methodologies remain limited. A recent contribution addressing this gap is the benchmark analysis framework proposed by Mitsopoulou and Koutrika [18]. Their study characterizes Text-to-SQL datasets along three complementary dimensions: structural and operator diversity of SQL queries, schema size and complexity of associated databases, and lexical-syntactic properties of natural-language questions. This unified descriptive perspective provides valuable insight into dataset composition, differences across benchmarks, and how dataset properties relate to model error profiles. The authors further introduce an "informative evaluation" procedure that decomposes system errors into structural,

operator, and variable mismatches, offering a more interpretable alternative to exact-match evaluation.

However, this analysis remains descriptive rather than diagnostic: it does not assess whether dataset entries are correct or semantically faithful. Several important aspects of dataset quality therefore fall outside its scope, including verification of schema integrity, checks of SQL syntactic and execution validity, and evaluation of semantic alignment between natural-language intent and SQL logic. Beyond surface-level schema linkage, the gold annotations are assumed to be correct, and no mechanisms are provided to detect mismatches between questions and queries. Moreover, the methodology is designed for aggregate statistics on existing benchmarks and does not scale to large modern datasets such as OmniSQL's 2.5M+ synthetic examples.

A complementary line of work is represented by NL2SQL-BUGs [17], which explicitly targets semantic correctness of gold SQL annotations in popular benchmarks. The authors construct an expert-annotated corpus of 2,018 NL–SQL pairs, of which 999 contain semantically incorrect SQL, organized into a detailed two-level taxonomy of semantic error types. This benchmark captures a rich variety of semantic bugs, including wrong aggregation functions, missing or superfluous filters, incorrect joins, and more subtle logical inconsistencies. In addition, the authors apply GPT-4o as an automatic semantic error detector to the development splits of Spider and BIRD, and, after manual verification of the flagged cases, identify previously unknown semantic errors in the gold SQL of both benchmarks. NL2SQL-BUGs thus provides strong evidence that widely used Text-to-SQL datasets contain semantic label noise and offers a valuable diagnostic resource, together with example prompts, for studying semantic error detection in NL2SQL translation. However, beyond the benchmark itself and these prompt templates, NL2SQL-BUGs does not contribute a reusable software framework or end-to-end toolchain for dataset auditing. Its scope is constrained to semantic mismatches between natural-language questions and SQL queries on selected subsets of existing benchmarks.

A different perspective on data quality is offered by Pandey et al. [19], who propose a "data accuracy" validation framework for enterprise Text-to-SQL systems. Their approach focuses on runtime validation of generated SQL and query results through schema validation, syntax checking, logical consistency checks (e.g., aggregate–grouping coherence), data integrity verification, and multi-agent feedback loops implemented with LangGraph and AutoGen. Although this framework uses several components like those needed for dataset quality assurance, it is fundamentally designed for per-query validation in production systems rather than large-scale offline auditing of benchmark or synthetic corpora. It operates on individual generated queries and their results and therefore does not analyze dataset-level error patterns, measure overall "dataset health", or provide tools for systematic dataset curation. Moreover, the framework is presented primarily at an architectural and conceptual level: there is no publicly available implementation, no quantitative evaluation on standard benchmarks (e.g., Spider, BIRD, OmniSQL), and no support for batch-mode processing of tens of thousands of NL–SQL pairs. Semantic checks are restricted to intra-SQL consistency (e.g., aggregates vs. GROUP BY) and do not address the correctness of the natural-language–to-SQL mapping across an entire dataset.

Additional efforts in the literature acknowledge the importance of validation but remain fragmented and narrowly scoped. Synthetic data pipelines (e.g., Gretel, OmniSQL) typically apply internal LLM-based or heuristic filters, while many works rely almost exclusively on execution-based filtering—discarding examples whose SQL fails to run on the corresponding database. Although such filtering can improve training stability, it is ultimately naïve and

insufficient: successful execution does not verify the correctness of SQL logic, the semantic alignment between the question and the query, the integrity of the underlying schema, or the presence of stylistic and efficiency-related anti-patterns. Consequently, these procedures cannot serve as general-purpose dataset auditing methods and address only a small subset of the quality issues that arise in modern Text-to-SQL datasets.

Taken together, existing approaches provide important analytical foundations and illustrate the need for quality control, but they do not offer a comprehensive, automated, and scalable solution for validating Text-to-SQL datasets themselves. Our work builds on these insights by introducing a multi-level validation framework—covering structural, syntactic, semantic, and stylistic dimensions—implemented in an extensible open-source system designed to evaluate both human-curated benchmarks and large-scale synthetic datasets at scale.

3. Methods

In this section, we describe the design, architecture, and implementation of our comprehensive Text-to-SQL dataset validation framework.

3.1. Architectural overview of the text2sql-dataset-analyzer framework

In this section we present `text2sql-dataset-analyzer`, a novel framework and, to the best of our knowledge, the first dedicated tool for systematic, multi-layer quality assessment of Text-to-SQL benchmarks. The framework is released as open-source software, which allows other researchers and practitioners to inspect, extend, and apply it to their own Text-to-SQL corpora.

Design principles. The system represents a configuration-driven streaming data-processing architecture designed to address the challenges of large-scale Text-to-SQL dataset validation and quality assessment. The architecture implements three foundational design principles: modularity through well-defined component interfaces, scalability via constant-memory streaming processing, and extensibility through configuration-driven composition. This design enables the system to process datasets of arbitrary size while maintaining deterministic memory footprint characteristics, a critical requirement when analyzing multi-gigabyte benchmark datasets such as Spider, BIRD, or OmniSQL 2.5 M.

Technology Stack. The implementation relies on a carefully selected technology stack centered on Python 3.11 [20], chosen for its mature ecosystem of SQL tooling and support for large language model integration. DuckDB [21] serves as the primary analytics backend, offering columnar storage optimized for analytical queries while exposing a standard SQL interface. For interaction with relational databases, we use SQLAlchemy 2.0 [22]–based adapters, which provide a uniform abstraction layer over concrete engines; in the current implementation, SQLite and PostgreSQL are supported as execution backends, but additional engines can be integrated with minimal effort. SQL parsing and structural analysis are handled by sqlglot [23], a dialect-aware parser capable of dealing with syntactic variations across SQLite, PostgreSQL, MySQL, and other SQL dialects commonly encountered in Text-to-SQL datasets. Component orchestration relies on the `dependency_injector` library [24], which facilitates type-safe dependency resolution and enables runtime composition of analysis pipelines through declarative YAML configuration loaded via PyYAML [25].

User Interface. User interaction with the system occurs primarily through a command-line interface exposed via the `text2sql` entry point. The interface supports two distinct operational modes corresponding to the system's primary use cases. The first mode, invoked through `text2sql run --config <config.yaml>`, executes the complete analysis pipeline from raw dataset ingestion through all configured analyzers to final metric persistence and report generation. The second mode, accessible via `text2sql report`, enables standalone report generation from previously computed metrics, supporting both configuration-driven batch report generation (`--config`) and targeted single-report generation (`--database <path> --type <report-type>`). This separation of concerns between analysis and reporting allows researchers to iterate on report designs and formats without re-executing computationally expensive analysis stages, particularly relevant when semantic validation via large language models can require hours of processing time.

3.2. Five-Layer System Architecture

The system architecture follows a layered design pattern comprising five runtime layers, all orchestrated by a separate configuration and orchestration layer that declaratively specifies components and their parameters (Figure 2). This layered organization promotes separation of concerns, facilitates independent testing and development of components, and enables selective activation or deactivation of functionality based on analysis requirements.

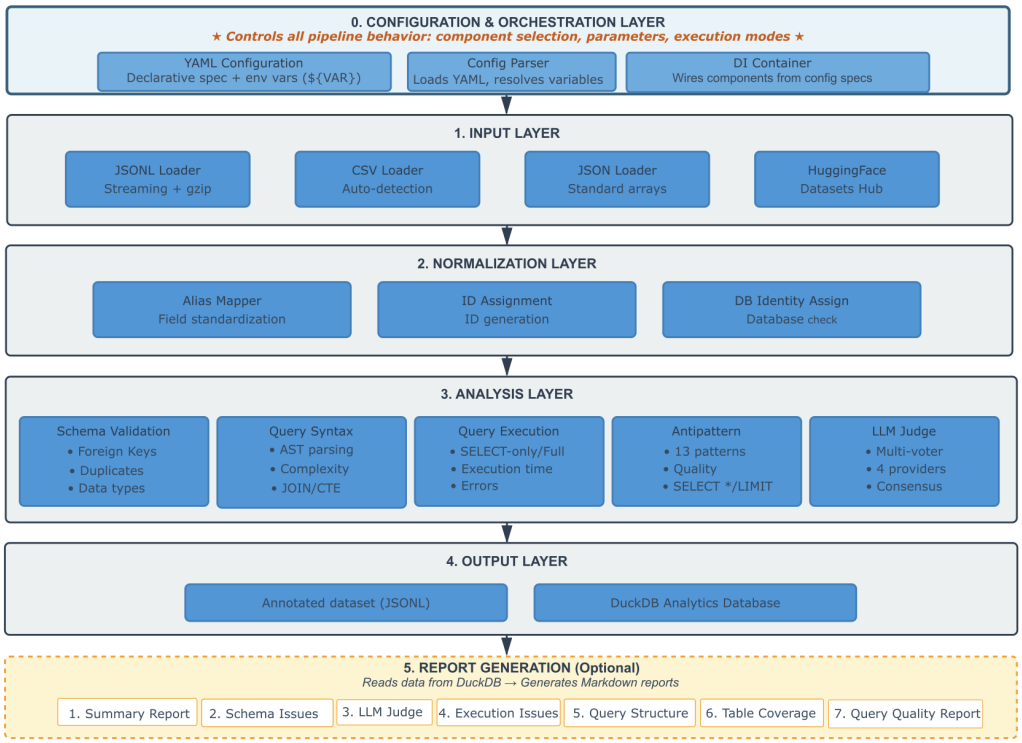

Figure 2. Five-layer architecture of the text2sql-pipeline system

Configuration & Orchestration. This cross-cutting foundation underpins all five runtime layers by declaratively specifying components, their parameters, and how they are wired together. A YAML-based configuration (with environment-variable interpolation such as `${VAR}`) captures all settings for loaders, normalizers, analyzers (e.g., antipattern penal-

ties), and output formats. A configuration parser loads and validates this specification, while a dependency-injection container instantiates and connects the corresponding components (e.g., `db_manager`), ensuring that the entire pipeline remains configuration-driven.

Input Layer. The Input Layer abstracts heterogeneous data sources behind a common iterator interface. It implements four specialized loaders tailored to data formats prevalent in Text-to-SQL research. The JSONL Loader provides streaming access to line-delimited JSON files, including gzip-compressed variants. The CSV Loader supports delimiter detection and encoding inference to handle noisy, real-world CSV files. Standard JSON arrays are handled by the JSON Loader, and the HuggingFace Loader connects directly to the HuggingFace Datasets Hub, avoiding manual download and reformatting. All loaders implement Python's iterator protocol and yield records on demand, keeping memory usage essentially constant with respect to dataset size.

Normalization Layer. The Normalization Layer standardizes records originating from diverse datasets through three sequential normalizers. The Alias Mapper reconciles different field names (e.g., `query` vs `sql`, `db_id` vs `dbId`) into a canonical schema. The ID Assignment component guarantees a unique identifier for each record using either counters or content-based hashes, supporting consistent cross-referencing between the annotated dataset and the metrics database. The DB Identity Normalizer ensures that each record is linked to a valid database: it validates provided database identifiers or, when necessary, builds databases from embedded DDL schemas and derives stable IDs from schema hashes. The result is a normalized DataItem with mandatory fields (`id`, `question`, `sql`, `dbId`), an optional schema, and a metadata dictionary.

Analysis Layer. The Analysis Layer is the computational core, hosting five analyzers that cover complementary quality dimensions of Text-to-SQL data:

1. **Schema Validation Analyzer** checks database integrity, including several classes of foreign-key violations, case-insensitive column name duplicates, and type compatibility with the target SQL dialect. A DB-ID-based cache reuses schema introspection results across queries sharing the same database.
2. **Query Syntax Analyzer** performs static analysis using the sqlglot parser. It builds abstract syntax trees, derives normalized complexity scores on a 0–100 scale based on SQL constructs and nesting depth, extracts structural features (e.g., JOIN counts, subquery depth, CTEs, aggregations, window functions), and assigns difficulty labels (Easy, Medium, Hard, Expert).
3. **Query Execution Analyzer** evaluates dynamic executability within the corresponding database. Queries are run in a sandboxed, SELECT-only mode; execution time and row counts are recorded, and failures are categorized to distinguish syntax problems, semantic errors, and connectivity issues.
4. **Query Antipattern Detector** uses a rule-based approach to flag 13 SQL antipatterns, such as functions in WHERE predicates blocking index usage, correlated subqueries with quadratic complexity, and implicit joins. Each detected antipattern contributes a weighted penalty to a 0–100 quality score.
5. **Semantic LLM Judge Analyzer** uses large language models to assess how well SQL queries answer their natural-language questions.

Output Layer. The Output Layer persists each analyzer's results through two complementary output streams. The Annotated Dataset augments each original record with all analyzer metadata and stores it in JSONL format, preserving a detailed provenance for downstream filtering or model training. In parallel, the DuckDB Analytics Database stores

structured metrics in a columnar OLAP database. Users can query it with SQL, export results to Pandas or Polars, or write Parquet files for long-term storage and integration with external analytics workflows. This dual strategy preserves both record-level detail and dataset-level statistics without re-running the pipeline.

Report Generation Layer (optional). The Report Generation Layer converts metrics stored in DuckDB into human-readable Markdown reports. Controlled via a `reports.enabled` configuration flag, it can be disabled during rapid experimentation on large datasets. When active, it produces seven report types: overall summary statistics; schema validation issues; semantic validation problems (queries judged incorrect or unanswerable); execution failures; structural complexity and difficulty distributions; table coverage analysis; and SQL quality assessments based on antipatterns. Because DuckDB is the single source of truth for metrics, reports can be regenerated or customized at any time via ad-hoc SQL queries, and report generation can be omitted entirely when performance constraints are critical.

3.3. Data Flow and Processing Model

Figure 3 shows the lifecycle of a single dataset record as it moves through the streaming pipeline. The system follows the iterator pattern: each record is processed end-to-end before the next one is loaded, without ever holding the full dataset in memory. Processing starts when a loader reads a raw record and yields it as a Python dictionary. The record then passes through the three normalizers described earlier. Each normalizer applies its transformation or validation and forwards the result, producing a standardized DataItem with all required fields present and a verified database identity. The normalized DataItem then enters the analysis pipeline, implemented as a chain of generator-based analyzers. Each analyzer consumes a DataItem, runs its logic (schema validation, syntax analysis, execution testing, antipattern detection, or semantic judging), writes metrics to DuckDB, enriches the DataItem's metadata, and yields it to the next analyzer. All configured analyses for one record complete before the next record is read, preserving strict streaming semantics. After the final analyzer, the fully annotated DataItem reaches the output layer. One branch serializes it to a JSONL file that contains the original fields plus all metadata, enabling downstream use without access to the metrics database. The other branch accumulates structured metrics in DuckDB with batched writes, supporting efficient aggregate analysis.

The last report layer, which can be disabled via configuration (`reports.enabled = false`), reads metrics from DuckDB and produces seven types of human-readable analytical reports in Markdown format.

3.4. Database Identity Normalization Algorithm

The Database Identity Normalizer addresses a fundamental challenge in Text-to-SQL dataset processing: ensuring that every record can be associated with a valid, accessible database instance against which SQL queries can be validated and executed. This challenge arises from the diversity of dataset formats and conventions employed across the Text-to-SQL research community, where some datasets provide explicit database identifiers referencing pre-existing database files (Spider and Bird datasets), others include DDL schemas allowing dynamic database construction, and some provide both, occasionally with inconsistencies between the identifier and schema.

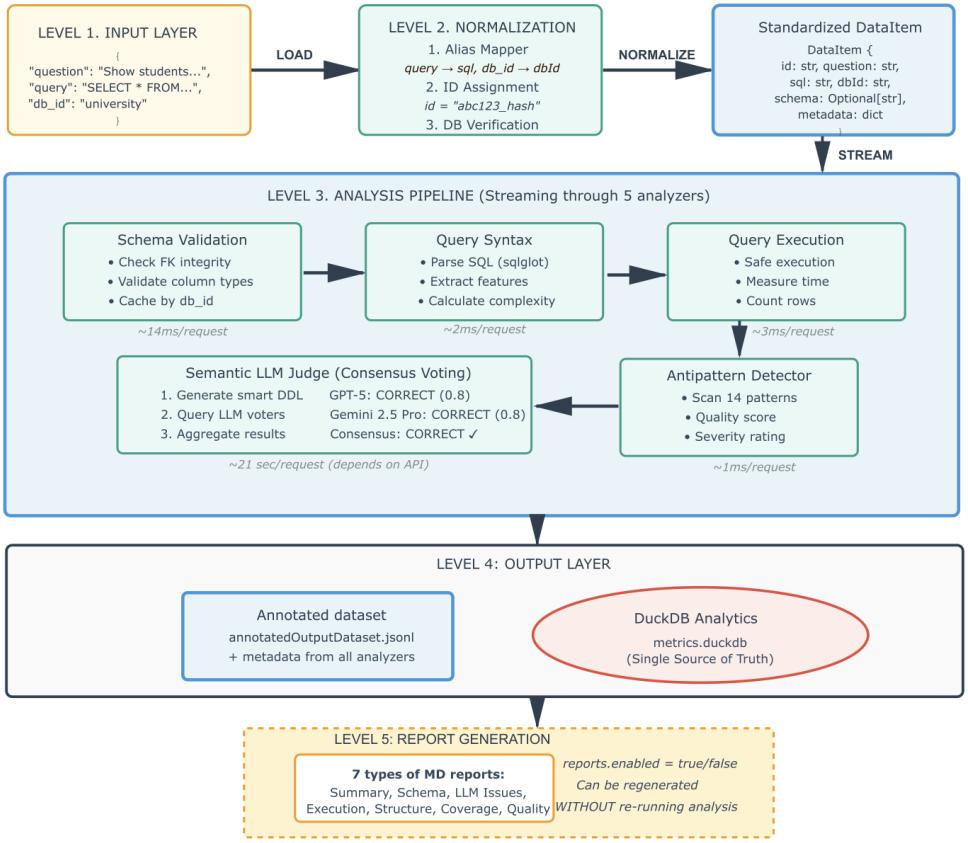

Figure 3. Data flow through the pipeline from input to output

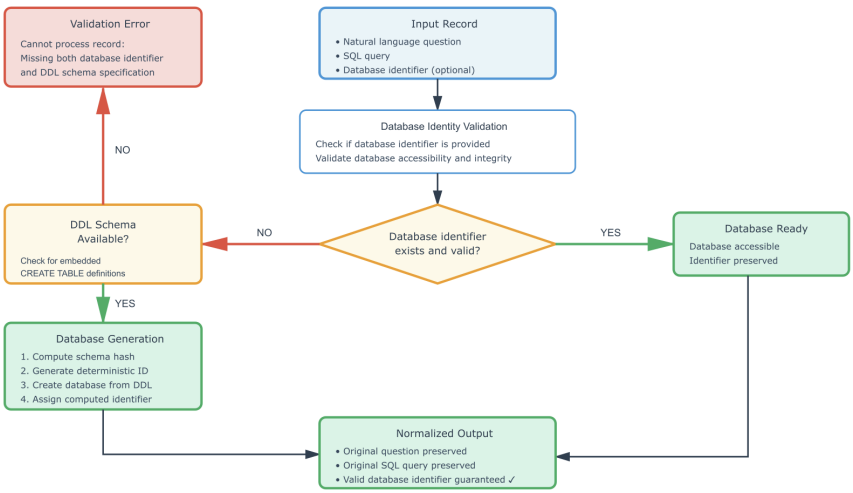

Figure 4. DbIdentity normalization process: guaranteed dbId filling via health check or creation from DDL

The algorithm implements a decision tree with three primary branches corresponding to different combinations of database identifier presence and schema availability (Figure 4).

In the first scenario, when a record provides an explicit database identifier, the normalizer invokes a health check through the database manager, querying whether the referenced database exists and contains data. If the check passes, the record proceeds unchanged; if it fails, the normalizer attempts to recreate the database from its stored DDL schema.

The second scenario handles records lacking explicit database identifiers but including DDL schema specifications, a pattern commonly encountered in synthetic datasets or benchmark subsets designed for testing schema understanding capabilities. For these records, the normalizer computes a cryptographic hash (SHA-256) of the DDL schema text, generating a deterministic identifier that remains consistent across multiple pipeline executions processing the same schema. The normalizer then checks whether a database with this computed identifier already exists; if so, it reuses that database instance, leveraging caching to avoid redundant database creation operations when multiple records share identical schemas. If no such database exists, the normalizer creates a new database from the DDL schema, assigns it the computed hash identifier, and populates the record's `dbId` field with this identifier. This deterministic identifier generation strategy ensures reproducibility while supporting the analysis of DDL-only datasets that lack pre-existing database files.

The third scenario represents failure cases where records provide neither a database identifier nor a DDL schema. In this situation, the normalizer raises a `ValueError` with a descriptive error message, immediately halting processing of that record. This fail-fast behavior reflects a deliberate design decision: rather than attempting graceful degradation by skipping validation steps that require database access, the system treats the absence of database identification as a fatal data quality issue that must be addressed at the source. This strict validation ensures that downstream analyzers can rely on the guaranteed presence of a valid database identifier, simplifying their implementation and preventing silent failures where analyses would be skipped due to missing database access.

3.5. Schema Validation Analyzer

The Schema Validation Analyzer constitutes a critical component of the quality assessment pipeline, scrutinizing database schema integrity to detect structural errors and data quality issues that could compromise query execution correctness or semantic validity. Unlike syntax analysis which examines SQL query correctness or execution testing which validates runtime behavior, schema validation focuses on the foundational layer—the database structure itself—identifying problems in table definitions, constraint specifications, and referential integrity that affect all queries operating against those databases. The analyzer implements a dual-level validation architecture distinguishing between structural errors in schema declarations (design flaws preventing proper constraint enforcement) and data violations in existing rows (integrity breaches where data contradicts declared constraints), enabling both proactive schema repair and targeted data cleaning strategies.

Figure 5 illustrates the three-branch schema validation architecture. The analyzer processes databases through parallel validation pathways, each targeting distinct quality dimensions. The first branch extracts foreign key definitions from database metadata through dialect-specific system catalogs, with fallback to dataset-provided schema files when explicit constraint metadata proves unavailable. The second branch performs schema introspection via column enumeration and primary key discovery to detect structural anomalies. The

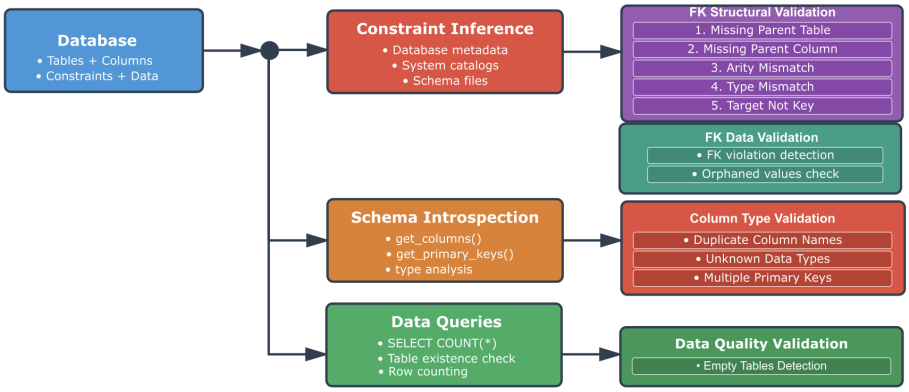

Figure 5. Schema validation analyzer architecture

third branch executes data queries to assess content quality. This multi-pathway design enables comprehensive validation spanning constraint integrity, column definitions, and data completeness.

Foreign key structural validation examines constraint definitions through five checks: Missing Parent Table (verifying referenced tables exist), Missing Parent Column (confirming parent columns presence), Arity Mismatch (enforcing equal column counts in composite keys), Type Mismatch (detecting incompatible types after normalization to dialect affinity families), and Target Not Key (verifying parent columns possess uniqueness constraints). Foreign key data validation detects orphaned values—rows where foreign key column values lack corresponding parent table entries—through dialect-specific integrity check commands. Column and type validation operates independently, detecting duplicate column names within tables (case-insensitive comparison), unknown or unsupported data types, and tables with multiple primary key definitions. Data quality validation executes row count queries to identify empty tables—schema structures lacking actual data—which can confound training algorithms or produce misleading evaluation results when queries execute successfully against vacant schemas.

3.6. Query Syntax Analyzer

The Query Syntax Analyzer performs static structural analysis of SQL statements, extracting syntactic features and computing complexity metrics without database execution. The analyzer processes queries through four stages: abstract syntax tree construction, structural feature extraction, complexity score calculation, and difficulty classification (Figure 6).

The Abstract Syntax Tree Construction stage employs sqlglot parser to transform SQL text into a hierarchical tree where nodes represent syntactic constructs—SELECT expressions, JOIN operations, WHERE predicates, subqueries, and CTEs. Parsing failures are captured with detailed error messages facilitating identification of malformed queries.

In the Structural Feature Extraction stage, the analyzer traverses the AST to compute a feature vector capturing syntactic properties relevant to query complexity. Five feature groups are extracted. Basic structure features count projections, quantify WHERE predicate complexity, and detect GROUP BY and ORDER BY clauses. Join and relation features count JOIN types (INNER, LEFT, . . .), enumerate distinct tables, and compute maximum join depth, capturing relational complexity. Advanced features measure subquery nesting depth, CTE usage, window functions (OVER clauses), and recursive CTEs. Aggregation features track aggregate functions (COUNT, SUM, . . .), HAVING clauses, and DISTINCT

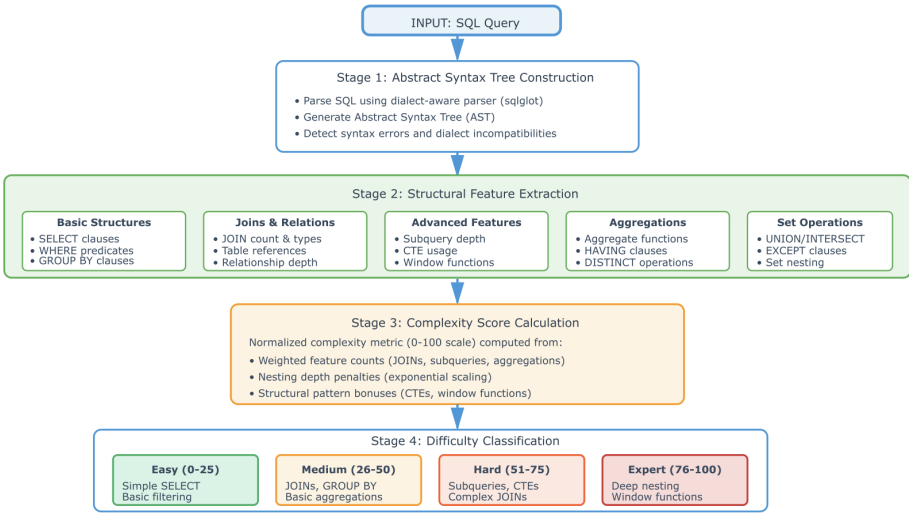

Figure 6. Query Syntax Analyzer architecture with four-stage processing pipeline

operations. Set operation features detect UNION, INTERSECT, and EXCEPT operators and their nesting patterns.

The Complexity Score Calculation stage synthesizes features into a 0–100 metric correlating with query difficulty. The algorithm implements weighted aggregation: single-table SELECT queries contribute 0–10 points, JOINs add 3–5 points, aggregates 2–4 points, and advanced features 5–10 points. Nesting depth penalties apply exponential scaling reflecting disproportionate cognitive costs, while well-structured CTEs and window functions receive bonuses. Raw scores undergo min-max normalization spanning 0–100.

The Difficulty Classification stage maps scores to four categorical labels with empirically calibrated thresholds. Easy queries (0–25) encompass single-table SELECT with simple predicates. Medium queries (26–50) introduce multi-table JOINs, GROUP BY, and straightforward subqueries requiring intermediate proficiency. Hard queries (51–75) exhibit multiple JOINs, nested subqueries, CTEs, and correlated subqueries challenging experienced practitioners. Expert queries (76–100) feature deep nesting, window functions, recursive CTEs, and intricate set operations representing frontier challenges. This enables benchmark coverage assessment, stratified evaluation, and imbalance identification.

3.7. Query Antipattern Detector

The Query Antipattern Detector implements a rule-based static analysis framework for identifying SQL constructs that compromise query correctness, data integrity, execution performance, or code maintainability. Unlike the Query Syntax Analyzer that quantifies structural complexity without normative judgment, antipattern detection applies domain-specific knowledge to identify patterns empirically associated with incorrect results, data corruption, or performance degradation. The detection methodology employs a three-stage analytical pipeline (Figure 7): AST-based pattern recognition through structural matching, dialect-configurable severity classification and weighted scoring.

Rule-Based Detection Engine. The detection engine implements 13 independent rules that traverse the sqlglot abstract syntax tree to identify antipattern signatures. Each rule operates independently, enabling parallel detection of multiple concurrent antipatterns. Detection leverages typed AST nodes (EQ, NEQ, AggFunc, Update, Delete, Where, Join,

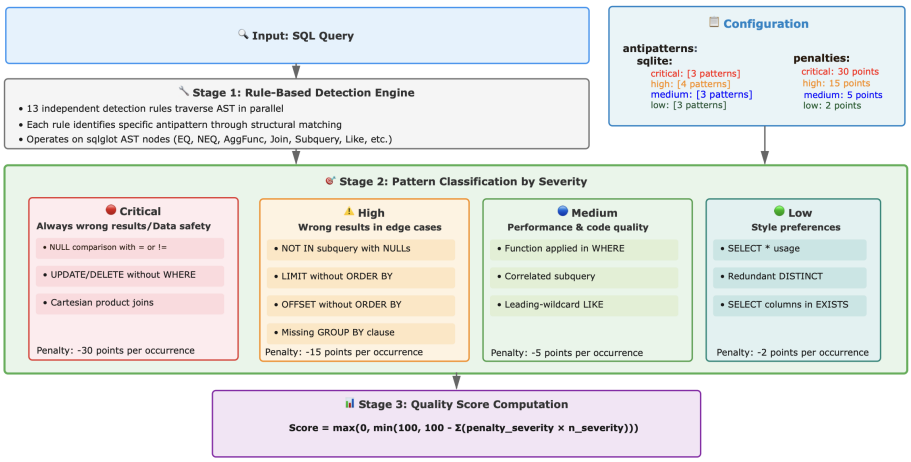

Figure 7. Query Antipattern Detector Architecture

Subquery, Like) to implement precise structural pattern matching across four quality dimensions: correctness violations, data safety issues, performance degradations, and code quality concerns.

Severity Classification. The system implements dialect-configurable severity assignment, acknowledging that antipattern impact varies across SQL implementations. Separate configuration mappings exist for SQLite and PostgreSQL, enabling context-appropriate assessment tailored to each database system's optimization capabilities and standard compliance. The following classification structure represents the SQLite-specific configuration used in our evaluation, with 13 patterns distributed across four severity levels.

Critical Severity (3 patterns) — Always produces wrong results or enables data corruption:

- **NULL comparison errors** (`col = NULL`): Equality operators with NULL always evaluate to NULL rather than true/false, breaking predicate logic [26, 27]. In SQL's three-valued logic system, NULL represents unknown values and comparing anything to unknown yields unknown (NULL) [27, 28]. Standard SQL requires IS NULL / IS NOT NULL predicates for NULL testing.
- **Unsafe UPDATE/DELETE**: Statements lacking WHERE clauses modify or delete all table rows, risking catastrophic data loss through unintended mass operations [26]. This is particularly dangerous in production environments where a single oversight can corrupt entire datasets.
- **Cartesian product joins**: Missing JOIN conditions cause exponential row multiplication ($N \times M$ rows from two N-row, M-row tables), producing massive unintended result sets that consume memory and processing resources [26, 33]. In most real queries, this indicates a logic error unless a CROSS JOIN was intentional.

High Severity (4 patterns) — Produces wrong results in specific edge cases:

- **NOT IN with nullable subqueries**: When subquery returns NULL, SQL's three-valued logic causes the entire NOT IN expression to evaluate to false/NULL unexpectedly, silently filtering all rows [26, 27]. Example: `WHERE id NOT IN (SELECT parent_id FROM table)` returns empty set if any `parent_id` is NULL.
- **LIMIT without ORDER BY**: Using LIMIT without ORDER BY, the database may returns an arbitrary subset of rows in an undefined order [26, 28]. The result set is

non-deterministic and may vary between executions, violating benchmark reproducibility requirements.

- **OFFSET without ORDER BY**: Similar to LIMIT without ORDER BY, OFFSET without ordering produces non-deterministic pagination [26,28]. Rows may shift between pages across executions, leading to duplicates or missing records in paginated results.
- **Missing GROUP BY**: Non-aggregated columns (bare columns) alongside aggregate functions without proper GROUP BY clause produce non-deterministic results. SQLite permits this behavior as an SQLite-specific extension, where bare column values are selected from an "arbitrarily selected row" within each group [29]. This affects all aggregate functions including COUNT(), SUM(), AVG(), MIN(), and MAX() [29]. While the SQLite documentation acknowledges this is intentional and states "this is considered a feature, not a bug" [29], this behavior deviates from the SQL standard [28] and is rejected by most other relational database systems including PostgreSQL, Oracle, and MySQL (when ONLY_FULL_GROUP_BY mode is enabled) [30, 31]. The non-deterministic nature of bare columns violates benchmark reproducibility requirements, as query results can vary between executions when multiple rows share the same GROUP BY key values.

Medium Severity (3 patterns) — Performance degradations and code quality issues:

- **Functions in WHERE** (`WHERE LOWER(col) = 'value'`): Applying functions to indexed columns prevents index usage, forcing full table scans [26, 32, 33]. The database cannot utilize the index because it must compute the function for every row before comparison.
- **Correlated subqueries**: Row-by-row execution exhibits $O(n^2)$ complexity [26, 32, 33]. Modern optimizers (PostgreSQL, Oracle) may decorrelate automatically, but SQLite executes literally, making this particularly problematic for large datasets [29].
- **Leading wildcard LIKE** (`LIKE '%pattern'`): Prevents index prefix matching, requiring full table scan even on indexed columns [26, 32, 33]. Indexes can only be used when the pattern starts with a known prefix.

Low Severity (3 patterns) — Style preferences acceptable in research contexts:

- **SELECT ***: Selects all columns implicitly, obscuring column-level dependencies and increasing coupling to schema changes, which negatively impacts readability and maintainability [26, 32]. In production code, explicit column selection is preferred for clarity and forward compatibility.
- **Redundant DISTINCT + GROUP BY**: GROUP BY already ensures uniqueness through aggregation; adding DISTINCT is redundant and may add unnecessary overhead [26, 32]. The duplicate elimination operation wastes CPU cycles and memory.
- **Columns in EXISTS subqueries** (`EXISTS (SELECT col FROM ...)`): EXISTS only checks row existence; selecting specific columns wastes I/O [26, 32]. The database may still fetch the column data unnecessarily. Using SELECT 1 or SELECT * is a convention that signals the columns are irrelevant.

Quality Score Computation. Detected antipatterns are synthesized into a scalar quality metric through weighted penalty aggregation. The general scoring formula is:

$$\text{quality_score} = 100 - \sum_{\text{severity}} n_{\text{severity}} \times p_{\text{severity}} \tag{1}$$

where n_{severity} denotes the frequency of antipatterns in each severity category, and p_{severity} represents the configurable penalty weight for that severity level. The penalty coefficients encode relative technical impact: correctness violations receive maximum penalty, edge-case failures moderate penalty, performance issues minor penalty, and style concerns minimal penalty.

It is worth noting that, although the Query Antipattern Detector is only one module of the overall system, it received the most extensive test coverage. Specifically, we implemented 267 tests that exercise all 13 detection rules and their dialect-configurable severity mappings. The suite targets high-risk SQL edge cases (e.g., NULL three-valued logic, omitted join predicates, non-deterministic pagination via LIMIT/OFFSET without ORDER BY, and aggregation corner cases), providing a regression harness against rule drift as the detector evolves.

3.8. Multi-Model Semantic Validation Architecture

The Semantic LLM Judge Analyzer constitutes the most sophisticated component of our validation pipeline, employing large language models to assess whether SQL queries correctly answer their associated natural language questions. This semantic validation capability addresses a critical limitation in traditional formal validation approaches: while syntax checking and execution testing verify technical correctness, only semantic analysis can determine whether a syntactically valid, successfully executing query implements the intended meaning of the natural language question. Our analyzer implements this capability through a four-stage pipeline combining intelligent context generation, prompt engineering, multi-model consensus voting, and result aggregation (Figure 8).

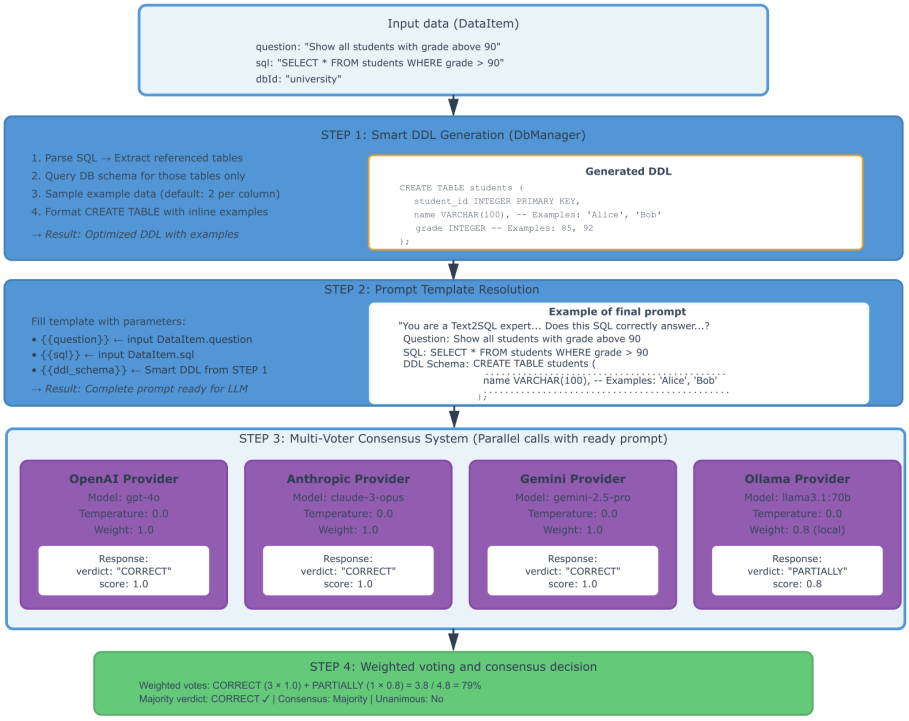

Figure 8. Architecture of the Semantic LLM Judge with multi-model consensus voting

Smart DDL Generation. The first stage addresses the token efficiency challenge inherent in providing database schema context to language models. Naïvely transmitting complete database schemas can consume thousands of tokens for databases with dozens of tables, rapidly exceeding context window limits or consuming substantial portions of available context that could be better utilized for reasoning. This stage prepares a compact, token-efficient representation of the database schema and sample data that is injected as context into the LLM-based semantic Text-to-SQL judge. The system supports two DDL generation modes: full and query-derived. In full mode, the system retrieves metadata for the entire database schema. In query-derived mode, it first parses the SQL query with sqlglot to build an abstract syntax tree and extract only the tables referenced in FROM clauses, JOIN conditions, and subqueries. The database adapter then queries the system catalogs only for these referenced tables, retrieving column names, data types, primary keys, and foreign key relationships. After this selection step, both modes follow the same pipeline: the system executes SELECT queries to sample a configurable number of example values from each column (by default, two per column), giving the language model concrete data instances that complement the type information. Finally, the collected metadata is formatted into CREATE TABLE statements with inline comments containing the example values, producing a minimal DDL specification.

In practice, query-derived mode reduces token usage by a factor of 2–3 compared to full mode, making it attractive for resource-constrained deployments; however, its truncated schema context and reliance on sparse value sampling can bias semantic evaluation, because the judge may lack the tables or representative examples needed to distinguish genuinely unanswerable questions from incorrect table selection. To avoid this and keep our semantic validation results cleanly interpretable, in this work we therefore report results using full mode (complete schemas), leaving a systematic comparison of query-derived and full mode, including a deeper analysis of value-sampling effects, to future work.

Prompt Template Resolution. The second stage combines the generated smart DDL with contextual elements to produce complete evaluation prompts. Templates are loaded from YAML configuration files to facilitate experimentation with different prompt formulations. Each template contains dynamic placeholders:

- {{`question`}}: The natural language question from the dataset, providing ground truth for query semantics
- {{`sql`}}: The SQL query under evaluation, formatted with syntax highlighting and normalized whitespace
- {{`ddl_schema`}}: The smart DDL from Stage 1, supplying necessary schema context
- {{`dialect`}}: Optional SQL dialect specification (sqlite, postgresql, etc.) for correct interpretation of dialect-specific syntax

The assembled prompt specifies a strictly structured response format in which the model must return a single JSON object with exactly two keys, “verdict” and “explanation” (Figure 9). The “verdict” field is constrained to one of four categorical labels: CORRECT (the query fully and accurately answers the question with no semantic issues), PARTIALLY_CORRECT (the query is mostly correct but exhibits minor deficiencies such as missing DISTINCT clauses when duplicates are possible, imprecise date/time boundaries, NULL-handling edge cases, or non-deterministic GROUP BY selections that do not fundamentally invalidate the result), INCORRECT (the query contains fundamental semantic errors including wrong table or column selections, missing or incorrect JOIN conditions, flawed aggregation logic, or erroneous WHERE clause predicates that produce

fundamentally incorrect results), and UNANSWERABLE (the question cannot be answered from the provided schema due to missing tables or columns, requirements for external data not present in the database, or inherent ambiguity or contradictions in the question itself).

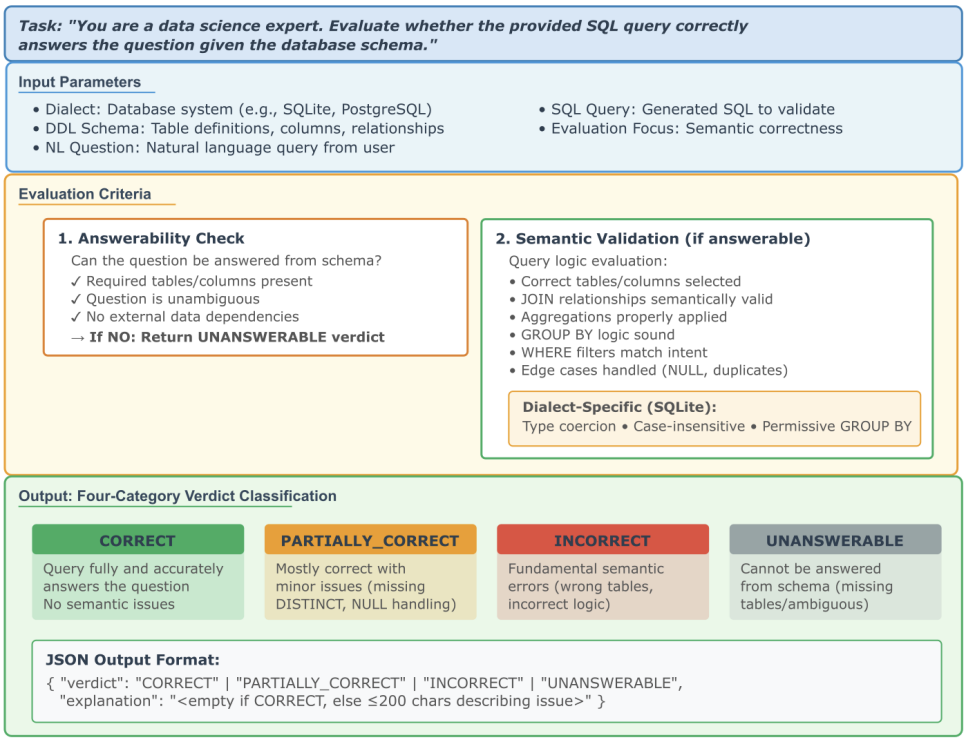

Figure 9. LLM prompt structure with input parameters, evaluation criteria, and structured response format with four-category verdict classification

The "explanation" field encodes a short natural-language justification; for CORRECT verdicts, the explanation is required to be an empty string, thereby preventing spurious narrative output, while for all other verdicts, the explanation must be a concise single-line description (bounded to at most 160–200 characters) that identifies the dominant semantic issue. This response specification transforms otherwise free-form LLM outputs into machine-parseable judgments that can be aggregated systematically across large datasets.

Multi-Model Consensus Voting. This stage queries multiple language models in parallel, collecting independent judgments. Our architecture supports four provider categories, each offering distinct characteristics:

- **OpenAI models** (GPT-5, GPT-4o, etc.) provide strong reasoning capabilities and reliable structured output generation, though with relatively high per-token costs.
- **Anthropic models** (Claude-4.5-sonnet, Claude-3-sonnet, etc.) offer alternative reasoning approaches that often exhibit different error patterns from GPT models, valuable for identifying edge cases where consensus breaks down.
- **Google models** (Gemini-2.5-pro, Gemini-1.5-pro, etc.) deliver competitive performance with distinct training data and architectural choices that surface different perspectives on ambiguous cases.
- **Ollama models** (Llama-3.1-70b, Mistral, Mixtral, etc.) enable fully local execution without API costs or rate limits, though typically with reduced accuracy compared to frontier commercial models.

This multi-model approach provides natural defense against systematic biases of individual models, where inter-model agreement serves as a reliability indicator for downstream analysis.

Consensus decision. The fourth stage aggregates the individual model responses through weighted voting to determine consensus verdicts and identify cases requiring human review. Each model is associated with a non-negative scalar weight specified in the configuration file, allowing researchers to encode prior beliefs about the relative reliability or cost-effectiveness of different providers (for example, assigning higher weights to empirically validated frontier models while giving lower weights to cheaper or fully local models). The aggregation procedure operates along two complementary axes. First, it computes an unweighted majority verdict by counting, for each of the four categories, how many models produced that verdict; a majority consensus is deemed to exist when a single category accounts for more than half of all non-failed model responses, and a verdict is unanimous when all such responses coincide. Second, it derives a scalar semantic quality score in the closed interval $[0, 1]$ by mapping CORRECT to 1.0, PARTIALLY_CORRECT to 0.5, and both INCORRECT and UNANSWERABLE to 0.0, and then computing a provider-weighted average according to formula:

$$\text{score}_{\text{consensus}} = \frac{\sum_{i=1}^{n} w_i \cdot v_i}{\sum_{i=1}^{n} w_i} \tag{2}$$

where v_i denotes the mapped verdict value and w_i the corresponding model weight. This score captures the aggregate tendency of weighted models to regard a query as semantically adequate, while the majority and unanimity indicators characterize the degree of inter-model agreement. Records lacking any strict majority or exhibiting strongly divergent verdicts across models are of particular interest for dataset curation, as they frequently correspond to questions whose intent is underspecified, to SQL queries with subtle semantic flaws, or to cases where multiple, mutually incompatible interpretations of the question are plausible.

The multi-model semantic validation architecture addresses several research challenges in Text-to-SQL evaluation. By employing multiple models with different architectures and training data, the system reduces the risk of systematic biases that might arise from relying on a single model family. The consensus voting mechanism provides natural uncertainty quantification, with disagreement among models serving as a signal that examples may be problematic or ambiguous rather than clearly correct or incorrect. The structured output format with confidence scores enables downstream analysis to weight semantic validation results appropriately, for example by flagging low-confidence verdicts for human review or by analyzing whether certain query patterns systematically produce low-confidence judgments.

4. Experimental Setup and Results

This section reports empirical results from applying the validation framework (Section 3) to all 11,840 examples in Spider 1.0 [7] across three partitions: Test (2,147 examples, 40 databases), Dev (1,034 examples, 20 databases), and Train (8,659 examples, 146 databases). The dataset was obtained from the official Spider resources maintained by the Yale LILY group [4] in January 2025. The experimental infrastructure employed the technology stack

described in Section 3, with SQLite 3.45.0 as the target dialect consistent with Spider's original database format [7, 34]. LLM-based semantic validation used OpenAI GPT-5 [35] and Google Gemini 2.5 Pro [36] (both accessed November 2025). These reasoning-optimized models were used under provider-default reasoning behavior.

The analysis pipeline operated in the streaming architecture described in Section 3, with analyzer components configured per the methodology detailed in Sections 3.5–3.8. Schema validation executed once per unique database identifier (206 total) with cached metadata reused for queries sharing databases. Query-level analyzers operated independently on each SQL statement. Antipattern detection followed the SQLite-specific severity configuration (Section 3.7) with 13 antipatterns and penalty weighting (Critical: -30, High: -15, Medium: -5, Low: -2). Execution testing used the query execution analyzer configured with `mode=all` (see Section 3), which executes SELECT statements with automatic LIMIT 1 injection. When a LIMIT clause is absent, it tests UPDATE/DELETE/INSERT statements inside a rollback transaction while blocking destructive operations (e.g., DROP/TRUNCATE/ALTER). Semantic judging used two-model consensus with verdict categories (CORRECT/PARTIALLY_CORRECT/INCORRECT/UNANSWERABLE) as defined in Section 3.8; configuration parameters included `schema_mode=full`, `num_examples=2`, and parallel voting enabled (`parallel_voters=true`, `max_workers=2`).

Results are organized across five quality dimensions reported in Sections 4.1–4.5: (i) database schema integrity (Section 4.1); (ii) SQL syntactic structure (Section 4.2); (iii) dynamic executability (Section 4.3); (iv) antipattern prevalence (Section 4.4); and (v) semantic correctness (Section 4.5). Sections 4.6–4.7 report computational performance and integrated quality assessment. Findings are presented descriptively without interpretation; Section 5 provides analysis of implications and validity threats.

4.1. Database Schema Integrity Assessment

Table 1 presents comparative schema validation results across all three Spider partitions.

Table 1. Comparative database schema validation results across Spider partitions

Metric	Spider Test	Spider Dev	Spider Train
Total databases	40	20	146
Valid databases	36 (90.0%)	13 (65.0%)	110 (75.3%)
Databases with errors	4 (10.0%)	7 (35.0%)	32 (21.9%)
Total tables	180	80	793
Empty tables	0 (0%)	0 (0%)	71 (9.0%)
Total foreign keys	146	64	717
Invalid foreign keys	4	7	78
Databases with FK data violations	1	3	6

The results reveal a clear quality gradient across partitions, with Test achieving 90.0% schema validity, Train at 75.3%, and Dev at 65.0%. This ordering contradicts the intuitive expectation that training data should exhibit higher quality than test data.

Foreign key structural errors total 89 across all partitions (4 in Test, 7 in Dev, 78 in Train). Table 2 presents the distribution of foreign key error types across partitions.

FK Type Mismatch (38 occurrences) and FK Target Not Key (43 occurrences) dominate structural errors. Referential integrity violations total 41,927 rows across 10 databases.

Table 3 enumerates specific database identifiers associated with each foreign key error type.

Table 2. Foreign key error distribution by type and partition

Error Type	Spider Test	Spider Dev	Spider Train	Total	Severity
STRUCTURAL ERRORS					
FK Missing Table	0	0	0	0	Critical
FK Missing Column	3	0	5	8	Critical
FK Arity Mismatch	0	0	0	0	Critical
FK Type Mismatch	1	5	32	38	High
FK Target Not Key	0	2	41	43	High
Duplicate Columns	0	0	0	0	Medium
Unknown Data Types	0	0	0	0	Low
Subtotal (Structural)	4	7	78	89	—
DATA VIOLATIONS					
FK Data Violations (rows)	1	2,403	39,523	41,927	Critical
Databases with Data Violations	1	3	6	10	—

Table 3. Database identifiers by foreign key error type (× denotes error count per database)

Error Type	Partition	Count	Database IDs
FK Missing Column	Test	3	book_1 (2×), car_racing (1×)
FK Missing Column	Dev	0	—
FK Missing Column	Train	5	baseball_1, imdb, loan_1, restaurants, store_product
FK Type Mismatch	Test	1	government_shift
FK Type Mismatch	Dev	5	car_1, concert_singer (2×), employee_hire_evaluation, museum_visit
FK Type Mismatch	Train	32	academic, aircraft (2×), architecture (2×), city_record, company_employee, cre_Drama_Workshop_Groups (9×), culture_company (2×), loan_1, machine_repair, party_people, performance_attendance (2×), phone_1, phone_market, race_track, school_finance (2×), shop_membership (2×), student_assessment, wrestler
FK Target Not Key	Test	0	—
FK Target Not Key	Dev	2	car_1, voter_1
FK Target Not Key	Train	41	baseball_1 (20×), dorm_1 (3×), imdb (6×), soccer_1 (4×), wine_1 (2×), yelp (7×)

Table 4 enumerates databases with foreign key data violations.

Table 4. Databases with foreign key data violations

Database ID	Partition	FK Data Violations	% of Total
sakila_1	Train	38,273	91.3%
flight_2	Dev	2,403	5.7%
flight_4	Train	1,240	3.0%
car_1	Dev	2	¡0.1%
hr_1	Train	6	¡0.1%
college_1	Train	2	¡0.1%
wta_1	Dev	1	¡0.1%
hospital_1	Train	1	¡0.1%
allergy_1	Train	1	¡0.1%
pilot_1	Test	1	¡0.1%
TOTAL	—	41,927	100%

Table 5 identifies databases with empty tables, found exclusively in the Train partition.

Table 5. Databases with empty tables in Spider Train partition

Database ID	Total Tables	Empty Tables	Data Coverage
academic	15	15	0%
imdb	16	16	0%
yelp	7	7	0%
restaurants	3	3	0%
geo	7	7	0%
music_2	7	7	0%
scholar	10	10	0%
sakila_1	16	4	75%
formula_1	13	2	85%
TOTAL (Train only)	94	71	24.5%

Seven databases exhibit 0% data coverage (65 empty tables), while sakila_1 and formula_1 show partial emptiness. Test and Dev contain no empty tables.

The Train partition exhibits an additional quality issue absent from Test and Dev: 41 tables across 4 databases (academic, imdb, yelp, restaurants) contain schema definitions but no data rows, representing empty tables that cannot support meaningful query evaluation. For instance, the `academic` database includes empty tables for conference proceedings and citations, preventing execution of queries that reference those tables. Similarly, the large-scale `imdb` database contains multiple empty junction tables that would normally link movies to actors, directors, and genres. While these empty tables do not technically violate schema correctness, their presence suggests incomplete data loading or extraction errors during dataset construction, and any queries in Spider Train that reference these tables will either fail to execute or return empty result sets regardless of query correctness.

4.2. Syntactic Structure and Complexity Analysis

Table 6 presents overall syntactic analysis results.

Table 6. Overall syntactic analysis results

Metric	Spider Test	Spider Dev	Spider Train
Total queries	2,147	1,034	8,659
Successfully parsed	2,147 (100%)	1,034 (100%)	8,659 (100%)
Parse failures	0 (0%)	0 (0%)	0 (0%)
Mean complexity score	39.5	39.0	40.7
Median complexity score	41	41	41
Standard deviation	20.2	19.9	21.2

All 11,840 queries parsed successfully (100%). Complexity scores show minimal variation: mean scores range from 39.0 to 40.7, with identical median (41) across all partitions.

Examination of difficulty level distributions (Table 7) reinforces this consistency pattern, with difficulty distributions consistent across Test and Dev, with approximately 24% Easy, 58–59% Medium, and 17% Hard. Train contains 66 Expert-level queries (0.8%), while Test and Dev contain none.

One notable exception to cross-partition consistency emerges in the Expert difficulty category: Train contains 66 Expert-level queries (0.8% of the partition) while Test and Dev contain none. These 66 queries represent the most structurally complex examples in the entire Spider dataset.

Table 7. Query difficulty distribution across partitions

Difficulty Level	Spider Test	Spider Dev	Spider Train	Complexity Range
Easy	516 (24.0%)	250 (24.2%)	2,039 (23.5%)	10–29
Medium	1,246 (58.0%)	612 (59.2%)	4,827 (55.7%)	30–59
Hard	385 (17.9%)	172 (16.6%)	1,727 (19.9%)	60–79
Expert	0 (0.0%)	0 (0.0%)	66 (0.8%)	80–100

Analysis of JOIN distribution (Table 8) reveals a second dimension where Train exhibits systematically higher complexity compared to Test and Dev. Single-table queries dominate all partitions (56–61%). Train exhibits higher multi-table query representation (44.02%) compared to Test (40.15%) and Dev (39.46%), with particularly elevated 3-JOIN and 4+-JOIN frequencies.

Table 8. JOIN distribution analysis

JOIN Count	Spider Test	Spider Dev	Spider Train
0 (single-table)	1,285 (59.85%)	626 (60.54%)	4,847 (55.98%)
1 JOIN	558 (25.99%)	320 (30.95%)	2,233 (25.79%)
2 JOINs	242 (11.27%)	72 (6.96%)	1,042 (12.03%)
3 JOINs	38 (1.77%)	10 (0.97%)	359 (4.15%)
4+ JOINs	24 (1.12%)	6 (0.58%)	178 (2.06%)
Multi-table ($\geq$1 JOIN)	862 (40.15%)	408 (39.46%)	3,812 (44.02%)

Despite the rich structural variation in JOIN complexity and overall difficulty levels, Spider exhibits complete absence of certain advanced SQL features. No queries in any partition employ common table expressions (WITH clauses), window functions (ROW_-NUMBER, RANK, LAG), or recursive queries.

4.3. Dynamic Executability Validation

Table 9 presents query execution results.

Table 9. Query execution results

Metric	Spider Test	Spider Dev	Spider Train
Total queries	2,147	1,034	8,659
Successfully executed	2,147 (100%)	1,034 (100%)	8,656 (99.97%)
Execution failures	0 (0%)	0 (0%)	3 (0.03%)
Failed items	—	—	3154, 4514, 4515
Skipped queries	0 (0%)	0 (0%)	0 (0%)
Mean execution time (ms)	3.10	2.49	1.19

Test and Dev achieve 100% execution success; Train achieves 99.97% (3 failures out of 8,659). Mean execution times remain under 3.5 milliseconds across all partitions.

4.4. Code Quality Assessment via Antipattern Detection

Table 10 presents overall code quality metrics.

The most of queries contain no detected antipatterns: 82.5% (Test), 83.7% (Dev), 86.5% (Train).

Table 11 presents antipattern distribution by type and severity.

Table 10. Overall code quality metrics

Metric	Spider Test	Spider Dev	Spider Train
Queries without antipatterns	1,772 (82.5%)	865 (83.7%)	7,494 (86.5%)
Total antipattern occurrences	383	169	1,182

Table 11. Antipattern distribution by severity

Antipattern	Spider Test	Spider Dev	Spider Train	Severity
Cartesian product	2 (0.1%)	2 (0.2%)	13 (0.2%)	Critical
Items with a Cartesian Product	945, 946	388, 389	3132, 3662, 3663, 5779, 5780, 6796, 6797, 6798, 6799, 6800, 6801, 6802, 6803	
Missing GROUP BY	239 (11.1%)	106 (10.3%)	600 (6.9%)	High
NOT IN with nullable columns	74 (3.4%)	46 (4.4%)	228 (2.6%)	High
Leading wildcard LIKE	50 (2.3%)	12 (1.2%)	140 (1.6%)	Medium
Function in WHERE clause	0 (0.0%)	0 (0.0%)	6 (0.1%)	Medium
Redundant DISTINCT	6 (0.3%)	0 (0.0%)	133 (1.5%)	Low
SELECT *	12 (0.6%)	3 (0.3%)	62 (0.7%)	Low

Missing GROUP BY is the most frequent high severity antipattern (945 total occurrences), affecting 6.9–11.1% of queries per partition. NOT IN with nullable columns affects 2.6–4.4% of queries. Critical Cartesian products total 17 queries across all partitions (2 in Test, 2 in Dev, 13 in Train). All 17 Cartesian product cases were manually verified and confirmed as incorrect queries representing broken join logic; notably, all 17 were also unanimously judged INCORRECT by both LLM models (Gemini 2.5 Pro and GPT-5) in semantic validation (Section 4.5), providing independent confirmation of their defect status.

4.5. Semantic Correctness via LLM Consensus

Table 12 presents semantic validation results obtained via LLM consensus voting (Gemini 2.5 Pro and GPT-5).

Table 12. Semantic validation results via LLM consensus voting

Consensus Verdict	Spider Test	Spider Dev	Spider Train
CORRECT	1,409 (65.6%)	684 (66.2%)	5,188 (59.9%)
PARTIALLY_CORRECT	43 (2.0%)	14 (1.4%)	259 (3.0%)
INCORRECT	171 (8.0%)	94 (9.1%)	867 (10.0%)
Mixed (disagreement)	522 (24.3%)	242 (23.4%)	2,316 (26.7%)
UNANSWERABLE	2 (0.1%)	0 (0.0%)	29 (0.3%)
UNANSWERABLE items	1129, 1265		155, 1017, 2534, 3037, 3038, 3394, 4393, 4926, 4955, 4956, 4972, 4990, 5012, 5173, 5174, 5352, 6582, 6583, 7295, 7299, 7301, 7394, 7399, 7487, 7488, 7489, 8022, 8024, 8025
TOTAL	2,147 (100%)	1,034 (100%)	8,659 (100%)

Test and Dev exhibit comparable correctness rates (65.6% and 66.2%), while Train shows lower correctness (59.9%). Explicitly incorrect queries total 1,132 across all partitions (8.0–10.0% per partition). Model disagreement (Mixed) affects 23.4–26.7% of queries across partitions.

Table 13 breaks down the Mixed category by disagreement direction (Gemini → GPT-5), with percentages computed relative to the Mixed subset.

Table 13. Mixed consensus breakdown (percentages relative to Mixed category)

Disagreement Pattern (Gemini → GPT-5)	**Spider Test**	**Spider Dev**	**Spider Train**
CORRECT → PARTIALLY_CORRECT	315 (60.3%)	144 (59.5%)	1,375 (59.4%)
CORRECT → INCORRECT	124 (23.8%)	57 (23.6%)	449 (19.4%)
INCORRECT → PARTIALLY_CORRECT	35 (6.7%)	17 (7.0%)	169 (7.3%)
INCORRECT → CORRECT	20 (3.8%)	14 (5.8%)	121 (5.2%)
CORRECT → UNANSWERABLE	14 (2.7%)	4 (1.7%)	94 (4.1%)
PARTIALLY_CORRECT → CORRECT	6 (1.1%)	1 (0.4%)	11 (0.5%)
PARTIALLY_CORRECT → INCORRECT	6 (1.1%)	2 (0.8%)	30 (1.3%)
INCORRECT → UNANSWERABLE	1 (0.2%)	3 (1.2%)	40 (1.7%)
UNANSWERABLE → CORRECT	0 (0.0%)	0 (0.0%)	14 (0.6%)
UNANSWERABLE → PARTIALLY_CORRECT	0 (0.0%)	0 (0.0%)	6 (0.3%)
PARTIALLY_CORRECT → UNANSWERABLE	0 (0.0%)	0 (0.0%)	5 (0.2%)
UNANSWERABLE → INCORRECT	1 (0.2%)	0 (0.0%)	2 (0.1%)

The dominant pattern is the CORRECT ↔ PARTIALLY_CORRECT boundary (∼59–60% of Mixed cases), consistent with instances where the generated SQL largely matches intent but differs in minor constraints (e.g., DISTINCT/NULL handling, small filtering nuances). The next-largest group is CORRECT ↔ INCORRECT disagreements (∼19–24% of Mixed), reflecting cases where judges differ in how strictly they interpret semantic mismatches. Importantly, disagreement patterns that involve an INCORRECT judgment (e.g., INCORRECT ↔ PARTIALLY_CORRECT at 6.7–7.3%, and INCORRECT ↔ UNANSWERABLE patterns in "Other") likely correspond to fundamentally flawed queries rather than benign ambiguity, even if the models disagree on severity. Under this interpretation, the effective incorrect rate may be higher than what unanimous INCORRECT verdicts alone indicate.

We manually reviewed all 31 cases unanimously labeled UNANSWERABLE and confirmed that the LLM consensus verdicts were correct: in each instance, the question cannot be answered from the provided schema alone (e.g., missing required tables/columns or inherent ambiguity). Finally, we cross-checked Dev-set semantic issues reported in [17] against our semantic validation. The 16 overlapping Dev items and our consensus outcomes were:

Consensus INCORRECT (9/16): 67, 494, 579, 773, 774, 819, 820, 961, 962

Consensus CORRECT (5/16): 101, 128, 129, 811, 812

Consensus Mixed (2/16): 555, 177 (both CORRECT↔PARTIALLY_CORRECT)

We manually reviewed all 16 items and confirmed that 101, 128, 129, 811, and 812 are semantically correct, despite being marked invalid in [17]. This suggests that some previously reported "semantic errors" reflect annotation noise or differing interpretation criteria, reinforcing the need for semantic evaluation protocols that explicitly accommodate ambiguity and multiple valid formulations.

4.6. Computational Performance Characteristics

Experiments executed on a single workstation (macOS 14.x, Apple Silicon, 16GB RAM). Table 14 presents per-query processing time for each analyzer component.

Table 14. Per-query processing time by analyzer component

Analyzer Component	Test (ms/query)	Dev (ms/query)	Train (ms/query)
Query Antipattern Detection	1.21	0.49	0.56
Query Syntax Analysis	2.32	0.60	0.64
Query Execution Testing	3.10	2.49	1.19
Schema Validation	14.25	16.91	9.93
Semantic LLM Judge	21,271	21,200	20,350

Formal analyzers (antipattern, syntax, execution, schema) complete within 20 milliseconds per query. The Semantic LLM Judge analyzer requires approximately 20–21 seconds per query, dominating overall runtime.

Table 15 presents aggregate processing time by partition.

Table 15. Aggregate processing time by partition

Component	Spider Test	Spider Dev	Spider Train
Fast analyzers (combined)	∼15 sec	∼8 sec	∼11 sec
LLM Judge	∼12.7 hr	∼6.1 hr	∼49.0 hr
I/O overhead	∼45 sec	∼22 sec	∼68 sec
Total pipeline time	∼12.7 hr	∼6.1 hr	∼49.0 hr
LLM fraction of total	99.9%	99.9%	99.9%

Total pipeline runtime is 12.7 hours for Test, 6.1 hours for Dev, and 49.0 hours for Train. LLM validation accounts for 99.9% of runtime across all partitions. Cumulative analysis time for all 11,840 examples totals approximately 67.8 hours.

4.7. Integrated Quality Assessment and Recommendations

Table 16 synthesizes key results across all validation dimensions for comparative assessment.

The integrated assessment reveals consistent patterns: all partitions achieve perfect or near-perfect parsing and execution, high code quality scores (97.7–98.4), but substantial schema defects (10–35% invalid databases) and moderate semantic correctness (60–66% correct). Dev exhibits the lowest schema validity (65.0%) despite its critical role in model selection. The 17 critical Cartesian product antipatterns represent confirmed incorrect queries, validated both through manual inspection and independent LLM semantic judgment.

Table 17 consolidates the most critical database quality issues identified, providing a prioritized remediation list for Spider maintainers.

Table 16. Integrated quality assessment across all validation dimensions

Quality Dimension	Spider Test	Spider Dev	Spider Train
DATABASE SCHEMAS			
Valid schemas (%)	90.0	65.0	75.3
Structural FK errors (count)	4	7	78
FK data violations (rows)	1	2,403	39,523
SYNTACTIC QUALITY			
Parse success (%)	100	100	100
Mean complexity score	39.5	39.0	40.7
EXECUTABILITY			
Execution success (%)	100	100	99.97
CODE QUALITY			
Critical antipatterns (count)	2	2	13
High-severity antipatterns (%)	14.5	14.7	9.5
SEMANTIC CORRECTNESS			
Correct (%)	65.6	66.2	59.9
Incorrect (%)	8.0	9.1	10.0
Disputed/Mixed (%)	24.3	23.4	26.7

Table 17. Critical databases requiring immediate remediation

Database ID	Partition	Critical Issues
sakila_1	Train	38,273 FK data violations + 4 empty tables
flight_2	Dev	2,403 FK data violations → EVALUATION BIAS
flight_4	Train	1,240 FK data violations
hr_1	Train	6 FK data violations
college_1	Train	2 FK data violations
car_1	Dev	2 FK data violations + structural FK errors
hospital_1	Train	1 FK data violation
allergy_1	Train	1 FK data violation
wta_1	Dev	1 FK data violation
pilot_1	Test	1 FK data violation → EVALUATION BIAS
book_1	Test	2 FK missing column
car_racing	Test	1 FK missing column
baseball_1	Train	FK missing column (Critical) + 20 FK target-not-key errors
imdb	Train	FK missing column (Critical) + 6 FK target-not-key errors + all 16 tables empty
loan_1	Train	1 FK missing column + 1 FK type mismatch
store_product	Train	FK missing column
restaurants	Train	FK missing column (Critical) + all 3 tables empty

5. Discussion

This section interprets the empirical findings reported in Section 4, examines their implications for Text-to-SQL research and practice, and discusses the validity threats and limitations of our study.

5.1. The Paradox of Formal Correctness and Hidden Defects

The results reveal a striking paradox: Spider achieves near-perfect scores on traditional quality metrics (100% parsing success, 99.97–100% execution success) while simultaneously exhibiting substantial structural and semantic defects. This finding challenges the implicit assumption in much Text-to-SQL research that successful query execution implies dataset

correctness. The 89 structural foreign key errors and 41,927 referential integrity violations documented in Section 4.1 remain invisible to standard evaluation protocols because SQLite does not enforce foreign-key constraints unless explicitly enabled, allowing queries to execute despite relational integrity violations [34]. This disconnect between execution success and data quality has significant implications: execution-based evaluation metrics (e.g., Execution Match) may report stable scores even when evaluating models against corrupted ground truth, leading researchers to trust benchmark results that may be systematically biased.

5.2. Partition Quality Heterogeneity and Evaluation Bias

The quality gradient observed across partitions—Test (90.0% valid schemas), Train (75.3%), Dev (65.0%)—contradicts standard assumptions about benchmark construction. We would expect the evaluation partitions (Test/Dev) to receive more rigorous quality assurance than the training partition, yet Dev exhibits the lowest schema validity. This finding is particularly concerning because Dev serves as the primary signal for hyperparameter tuning and model selection in typical Text-to-SQL research workflows [3, 7]. The presence of 2,403 referential integrity violations in Dev's flight_2 database (Section 4.1, Table 17) affects approximately 35.6% of Dev examples, potentially introducing systematic bias into validation metrics that researchers use to select between competing approaches. Models optimized against this corrupted validation signal may exhibit degraded performance on clean data while appearing superior during development, leading to suboptimal architecture choices and misleading reported improvements.

5.3. Antipattern Prevalence and SQL Portability

The antipattern analysis (Section 4.4) reveals that specific high-severity patterns exhibit concerning prevalence. Missing GROUP BY affects 945 queries (8.0% of the dataset), exploiting SQLite's permissive handling of aggregates with non-aggregated "bare" columns—behavior that is non-portable and can fail under stricter engines or modes (e.g., PostgreSQL, and MySQL when ONLY_FULL_GROUP_BY is enabled) [29–31]. Models trained on these examples may learn dialect-specific SQL that breaks when deployed against stricter systems. Similarly, 348 queries use NOT IN with potentially nullable columns, introducing correctness risks under SQL's three-valued logic: if the subquery returns NULL, the predicate can evaluate to UNKNOWN (and thus be filtered out in WHERE), removing rows unexpectedly [28]. These patterns matter less for benchmark performance (where evaluation uses the same SQLite environment that training examples target) but become critical in production environments with heterogeneous database systems and strict SQL conformance requirements. Finally, the critical Cartesian-product join cases identified in Section 4.4 represent a particularly severe portability and correctness hazard because they can silently inflate result sets and teach pathological join behavior.

5.4. Semantic Ambiguity and the Limits of Automated Evaluation

The semantic validation results (Section 4.5) provide direct evidence that formal correctness is insufficient for evaluating Text-to-SQL systems. Only 60–66% of examples achieved unanimous CORRECT verdicts from two frontier LLM judges, while 8–10% were unanimously judged INCORRECT, and 23–27% fell into the Mixed disagreement category. The disagreement breakdown shows that Mixed cases concentrate primarily at

the CORRECT↔PARTIALLY_CORRECT boundary (∼59–60% of Mixed), indicating that many disputed examples admit multiple valid SQL implementations or differ in subtle constraints (e.g., DISTINCT, NULL handling, boundary conditions). These boundary cases expose a fundamental challenge for benchmark-based evaluation: exact-match metrics penalize plausible alternatives, while execution-based metrics may accept incorrect solutions that coincidentally match results on small test databases.

Importantly, Mixed does not always reflect benign ambiguity. A substantial fraction of Mixed outcomes include an INCORRECT judgment (e.g., CORRECT↔INCORRECT at ∼19–24% of Mixed, and INCORRECT↔PARTIALLY_CORRECT at ∼7%), suggesting that the effective error rate may exceed what unanimous INCORRECT verdicts alone indicate. At the same time, not all outcomes are ambiguous: we manually reviewed all 31 cases unanimously labeled UNANSWERABLE and confirmed the consensus verdicts (Section 4.5). Similarly, we manually verified all 17 critical Cartesian-product join cases and confirmed they are genuine defects; both judges unanimously labeled all 17 as INCORRECT (Section 4.4). Finally, our cross-check against the 16 Dev-set semantic issues reported in [17] shows that some items labeled "invalid" in prior analyses are semantically correct under manual review, reinforcing that benchmark "correctness" depends on interpretation criteria and motivating evaluation protocols that explicitly accommodate ambiguity and multiple valid formulations.

5.5. Remediation Priorities for Spider

Based on impact analysis, we recommend the following remediation priorities for Spider 1.0: (i) CRITICAL: repair Dev's flight_2 database (2,403 violations affecting 35.6% of validation examples) before further benchmark use in model selection or published evaluations; (ii) HIGH: address Train's catastrophic databases (sakila_1 with 38,273 violations; flight_4 with 1,240 violations) to reduce training noise and prevent models from learning to accommodate integrity violations; (iii) MEDIUM: fix structural foreign key errors in 19 databases with FK Type Mismatch or FK Target Not Key errors; (iv) LOW: resolve or document the seven databases with 0% data coverage, either by restoring missing data or labeling them as schema-only examples unsuitable for execution-based evaluation.

Beyond database-level repair, maintainers should prioritize query-level curation by (v) fixing/removing verified high-severity defects (e.g., critical Cartesian-product joins) and (vi) explicitly annotating/excluding confirmed invalid items from standard accuracy reporting—including both non-solvable UNANSWERABLE questions and queries that are strongly supported as incorrect by multiple signals (agreement between both LLM judges, manual review, and prior semantic issue reports such as [17]). Finally, we recommend publishing an official high-confidence subset excluding examples with schema defects, confirmed invalid items, and high-severity query defects, enabling controlled experiments on known-clean data.

5.6. Threats to Validity

LLM-based semantic validation can introduce systematic biases. First, evaluation leakage/contamination is possible: frontier models may have been exposed to Spider or related artifacts during pre-training, which could influence judgments. While the substantial disagreement rate (23–27% Mixed verdicts) and non-trivial unanimous INCORRECT fraction (8–10%) suggest judges are not merely reproducing fixed labels, these observations

do not rule out contamination; eliminating this threat would require models demonstrably untrained on Spider.

Second, two-model consensus improves robustness relative to a single judge but does not remove correlated error: both models may share failure modes on SQL semantics (e.g., NULL behavior, duplicate elimination, boundary conditions) or specific domains, yielding high-confidence but still incorrect judgments. Accordingly, we treat consensus outcomes as strong signals rather than definitive ground truth and complement them with targeted manual checks on selected consensus subsets (Section 4.5). Third, providing sampled cell values (Section 3.8) introduces sampling bias: samples may miss edge cases (e.g., rare categories, extreme values, NULL prevalence) needed for accurate semantic judgment—especially for aggregates or distribution-dependent predicates—potentially leading to misclassification even when SQL is logically correct.

6. Conclusions and Future Work

This section summarizes the main findings and outlines future work.

6.1. Summary of Contributions

This paper presents `text2sql-dataset-analyzer`, an open-source, end-to-end framework for auditing Text-to-SQL datasets across complementary dimensions: database schema integrity, SQL syntactic structure and complexity, dynamic executability, rule-based antipattern detection, and semantic correspondence verification. The pipeline is reproducible and produces both dataset-level summaries and per-item annotations that support triage, curation, and longitudinal regression tracking.

Applying the framework to all 11,840 Spider 1.0 examples (Train/Dev/Test) shows that traditional checks can substantially overestimate dataset health. Despite 100% parsing success and 99.97–100% execution success, schema analysis uncovers 89 structural foreign-key errors and 41,927 referential-integrity violations concentrated in a small number of databases. We further observe pronounced quality heterogeneity across partitions: Dev exhibits the lowest schema validity (65.0%) even though it is the primary signal for model selection, and the flight_2 database alone affects 35.6% of Dev examples—a mismatch that can systematically bias hyperparameter tuning and validation-based model selection.

Semantic validation with a two-model LLM committee indicates that only 60–66% of items receive unanimous CORRECT judgments, while 8–10% are unanimously INCORRECT and 23–27% remain disputed. We ground these signals with targeted manual review. All 31 unanimously UNANSWERABLE cases were confirmed as non-solvable under the provided schemas. In addition, the Query Quality Analyzer flagged 17 high-severity Cartesian-product join cases (missing join predicates); all 17 also received an INCORRECT consensus from the semantic committee, and manual inspection verified them as real bugs. We also manually cross-checked 16 Dev semantic issues previously reported in [17] and found partial agreement: several items labeled as invalid in [17] are semantically correct under our criteria, highlighting that validity judgments depend on explicit interpretation rules (e.g., under specification, acceptable reformulations, and schema assumptions). These findings motivate publishing curated subsets together with transparent, testable labeling criteria.

Beyond correctness labels, the audit surfaces systematic portability and robustness hazards in the gold SQL, including missing GROUP BY under SQLite's permissive aggregate handling and NULL-sensitive NOT IN patterns. Taken together, the results support treating Text-to-SQL benchmarks as software artifacts that require continuous quality assurance, and they provide a prioritized repair roadmap for Spider focusing on critical Dev remediation (flight_2), high-impact Train repairs (e.g., sakila_1 and structural foreign-key issues), and semantic filtering strategies for high-confidence evaluation.

6.2. Limitations and Future Work

Full semantic validation remains expensive: end-to-end judging over 11,840 examples required approximately 67.8 hours and was dominated by API-based LLM calls. Future work should investigate cost-efficient strategies such as active-learning triage to prioritize uncertain cases, distilling an inexpensive semantic classifier from LLM-labeled data, and systems-level optimizations (higher parallelism under rate limits, caching, and incremental re-validation after dataset edits).

We validate only Spider 1.0; applying the framework to additional benchmarks and large synthetic corpora (e.g., BIRD, OmniSQL) would clarify which defect classes are dataset-specific versus systematic. The semantic committee currently uses two judges; extending to a more diverse set of models and incorporating stratified human spot checks could improve reliability, particularly when prior analyses (e.g., [17]) disagree due to differing criteria. Future work should also refine semantic judging context with richer but token-efficient database summaries (e.g., stratified value sampling and lightweight column statistics such as NULL rates and distinct counts) to reduce sampling-induced misclassification.

Finally, this study focuses on detection and prioritization rather than automated repair and downstream impact. Future work should develop repair pipelines for common defect classes (e.g., foreign key mismatches, missing GROUP BY rewrites, NULL-safe anti-joins) and evaluate portability by re-executing queries across stricter engines while reporting cross-dialect discrepancies. Controlled training studies comparing models trained and evaluated on the original dataset versus cleaned or high-confidence subsets are needed to quantify how dataset remediation affects generalization and robustness.

Overall, our results argue for treating Text-to-SQL benchmarks as software artefacts that require continuous quality assurance. By making comprehensive auditing practical and reproducible, the framework enables benchmark maintainers to track regressions, publish curated subsets, and prioritize repairs that measurably improve both training signal quality and evaluation validity.

Acknowledgment

The authors acknowledge the support of Intellias in funding and providing resources that enabled access to frontier large language models used in this study.

CRediT authorship contribution statement

Volodymyr Sabadosh: Conceptualization, Software Design, Data curation, Formal analysis, Methodology, Investigation, Visualization, Writing – original draft, Writing – review and editing. Vladyslav Kotsovsky: Supervision, Writing – review & editing.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The Text-to-SQL dataset analysis tool developed in this study is available as open-source software on GitHub: https://github.com/vsabadosh/text2sql-dataset-analyzer. The repository includes the configuration files and scripts required to reproduce the analysis, along with precomputed metrics and generated reports for the Spider 1.0 dataset. Detailed, partition-specific outputs (Train/Dev/Test), including the exported reports and artifacts referenced in the paper, are provided in the repository root under the MainSpiderResults directory.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial or not-for-profit sectors.

References

[1] J. Fürst, C. Kosten, F. Nooralahzadeh, Y. Zhang, and K. Stockinger, "Evaluating the data model robustness of text-to-sql systems based on real user queries," *CoRR*, Vol. abs/2402.08349, 2024. [Online]. https://arxiv.org/abs/2402.08349

[2] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, "A survey on employing large language models for text-to-sql tasks," *ACM Computing Surveys*, 2025, preprint: CoRR abs/2407.15186, doi:10.48550/arXiv.2407.15186. [Online]. https://doi.org/10.1145/3737873

[3] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian et al., "Text-to-sql empowered by large language models: A benchmark evaluation," *CoRR*, Vol. abs/2308.15363, 2023. [Online]. https://arxiv.org/abs/2308.15363

[4] Yale LILY Group, "Spider 1.0 — leaderboard," 2025, accessed: 2025-11-08. [Online]. https://yale-lily.github.io/spider

[5] V. Shkapenyuk, D. Srivastava, T. Johnson, and P. Ghane, "Automatic metadata extraction for text-to-sql," *CoRR*, Vol. abs/2505.19988, 2025. [Online]. https://arxiv.org/abs/2505.19988

[6] "Bird — leaderboard," 2025, accessed: 2025-11-08. [Online]. https://bird-bench.github.io/

[7] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 3911–3921. [Online]. https://aclanthology.org/D18-1425/

[8] Snowflake Inc., "Using snowflake copilot inline," Snowflake Documentation, 2025, accessed: 2025-11-08. [Online]. https://docs.snowflake.com/en/user-guide/snowflake-copilot-inline
[9] Microsoft, "What is an ai/bi genie space," Azure Databricks Documentation, 2025, accessed: 2025-11-08. [Online]. https://learn.microsoft.com/en-us/azure/databricks/genie/
[10] Microsoft, "Copilot in fabric in the sql database workload," Microsoft Learn, 2025, accessed: 2025-11-08. [Online]. https://learn.microsoft.com/en-us/fabric/database/sql/copilot-sql-database
[11] S. Chang, J. Wang, M. Dong, L. Pan, H. Zhu et al., "Dr.spider: A diagnostic evaluation benchmark towards text-to-sql robustness," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. https://arxiv.org/abs/2301.08881
[12] Z. Yao, Ł. Borchmann, B. Zhai, H. Zhang, and Y. He, "Arctic-text2sql-r1: Simple rewards, strong reasoning in text-to-sql," *CoRR*, Vol. abs/2505.20315, 2025. [Online]. https://arxiv.org/abs/2505.20315
[13] V. Zhong, C. Xiong, and R. Socher, "Seq2sql: Generating structured queries from natural language using reinforcement learning," *CoRR*, Vol. abs/1709.00103, 2017. [Online]. https://arxiv.org/abs/1709.00103
[14] J. Li, B. Hui, G. Qu, J. Yang, B. Li et al., "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," *CoRR*, Vol. abs/2305.03111, 2023. [Online]. https://arxiv.org/abs/2305.03111
[15] Gretel.ai, "Synthetic text-to-sql dataset (gretel-synth)," Hugging Face Datasets, 2024, accessed: 2025-11-08. [Online]. https://huggingface.co/datasets/gretelai/synthetic_text_to_sql
[16] H. Li, S. Wu, X. Zhang, X. Huang, J. Zhang et al., "Omnisql: Synthesizing high-quality text-to-sql data at scale," *CoRR*, Vol. abs/2503.02240, 2025. [Online]. https://arxiv.org/abs/2503.02240
[17] X. Liu, S. Shen, B. Li, N. Tang, and Y. Luo, "Nl2sql-bugs: A benchmark for detecting semantic errors in nl2sql translation," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*, 2025, pp. 5662–5673. [Online]. https://doi.org/10.1145/3711896.3737427
[18] A. Mitsopoulou and G. Koutrika, "Analysis of text-to-sql benchmarks: Limitations, challenges and opportunities," in *Proceedings of the 28th International Conference on Extending Database Technology (EDBT 2025)*. OpenProceedings.org, 2025, pp. 199–212. [Online]. https://doi.org/10.48786/edbt.2025.16
[19] P. Pandey, D. Patel, S. Mandvikar, and N. Kota, "Ensuring data accuracy in text-to-sql systems: A comprehensive validation framework," *International Journal of Computer Trends and Technology*, Vol. 72, No. 12, 2024, pp. 17–24. [Online]. https://doi.org/10.14445/22312803/IJCTT-V72I12P103
[20] Python Software Foundation, "Python 3.11 documentation," 2025, accessed: 2025-11-08. [Online]. https://docs.python.org/3.11/
[21] DuckDB Foundation, "Duckdb documentation," 2025, accessed: 2025-11-08. [Online]. https://duckdb.org/docs/stable/
[22] M. Bayer et al., "Sqlalchemy 2.0 documentation," 2025, accessed: 2025-11-08. [Online]. https://docs.sqlalchemy.org/en/20/
[23] T. Mao, "Sqlglot: Python sql parser and transpiler," 2025, accessed: 2025-11-08. [Online]. https://github.com/tobymao/sqlglot
[24] R. Mogylatov, "Dependency injector: Dependency injection framework for python," 2025, accessed: 2025-11-08. [Online]. https://python-dependency-injector.ets-labs.org/
[25] PyYAML Developers, "Pyyaml — yaml parser and emitter for python," 2025, accessed: 2025-11-08. [Online]. https://pyyaml.org/
[26] B. Karwin, *SQL Antipatterns: Avoiding the Pitfalls of Database Programming*. Pragmatic Bookshelf, 2010.
[27] C.J. Date, *SQL and Relational Theory: How to Write Accurate SQL Code*, 2nd ed. O'Reilly Media, 2012.
[28] International Organization for Standardization, "Iso/iec 9075-2:2016 information technology — database languages — sql — part 2: Foundation (sql/foundation)," Standard, 2016.

[29] SQLite, "Select — section 2.5: Bare columns in an aggregate query," 2025, accessed: 2025-11. [Online]. https://sqlite.org/lang_select.html#bare_columns_in_an_aggregate_query
[30] PostgreSQL Global Development Group, "The group by and having clauses," 2024, accessed: 2025-11. [Online]. https://www.postgresql.org/docs/current/queries-table-expressions.html#QUERIES-GROUP
[31] Oracle Corporation, "Mysql handling of group by," 2024, accessed: 2025-11. [Online]. https://dev.mysql.com/doc/refman/8.0/en/group-by-handling.html
[32] A. Molinaro and R. de Graaf, *SQL Cookbook: Query Solutions and Techniques for All SQL Users*, 2nd ed. O'Reilly Media, 2020.
[33] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.
[34] SQLite, "Foreign key support," 2025, accessed: 2025-11. [Online]. https://sqlite.org/foreignkeys.html
[35] OpenAI, "Models documentation (api reference)," 2025, accessed: 2025-11. [Online]. https://platform.openai.com/docs/models
[36] Google, "Gemini api documentation," 2025, accessed: 2025-11. [Online]. https://ai.google.dev/gemini-api/docs

Appendix A. Declaration of Generative AI Use

During the preparation of this manuscript, the authors used ChatGPT (OpenAI) as a supportive tool for language refinement and clarity. In addition, the authors used Cursor with an AI coding assistant (Claude Sonnet) to support implementation tasks (e.g., code suggestions and refactoring) in the development of the proposed framework. All generated suggestions were reviewed, tested, and, where applicable, manually revised by the authors. The authors take full responsibility for the content of the manuscript and the correctness of the software and results reported in this paper.

Authors and affiliations

Volodymyr Sabadosh
e-mail: vsababadosh@gmail.com
ORCID: https://orcid.org/0009-0006-9933-444X
Your Institution, Your Country

Vladyslav Kotsovsky
e-mail: vladyslav.kotsovsky@uzhnu.edu.ua
ORCID: https://orcid.org/0000-0002-7045-7933
Uzhhorod National University, Ukraine